

Thmanyah broadcast infrastructure implementation report

Prepared for the infrastructure engineering assessment. Validation date: 30 September 2026. The implementation and recorded tests run on an Apple Silicon Mac using Docker Desktop's Linux virtual machine.

The submission includes local application, monitoring, storage and streaming demonstrations, plus a short-lived AWS acceptance deployment. Real cloud checks covered a multipart S3 copy, three-VPC networking and VPN access, public application read/write, Ansible idempotence, database restore, encrypted SMB in both directions, and a 1 TiB logical sparse file read by Windows. The Windows drive mapping did not reconnect after reboot, and an OBS-to-MediaLive broadcast and S3 archive were not completed. All temporary AWS resources were removed after testing.

1 Requirement coverage and validation status

Requirement	Delivered	Validation performed
Windows volume mounted on Linux and reverse direction	SMB design, Windows scripts, Linux mount/fstab instructions, Samba lab	AWS Windows-to-Linux and Linux-to-Windows authenticated, encrypted SMB 3.1.1 read/write passed; 1 TiB sparse-file logical length passed; Windows drive reconnect after reboot failed
Filesystem supporting 1 TB per file	XFS loop-image demonstration and NTFS guidance	1 TiB sparse file created on XFS; real allocated-capacity test not performed
Monitoring and service alerts	Share probe, node exporter, Prometheus rules	Write/fsync/read probe, rule validation, alert firing and resolution executed; external paging not configured
Docker application with isolated communication	Nginx/frontend, Python API, PostgreSQL, secrets, health checks and limits	Read/write, process crash, recreation, DB outage, volume persistence, OOM containment and backup restore executed
S3 bucket-to-bucket multipart transfer	Python copy engine and Node-RED flow	Three-part AWS multipart copy and SHA-256 equality passed; objects and buckets deleted after test
Terraform and Ansible with three VPCs and VPN	AWS resources, inventory, vault templates, WireGuard and service configuration	Temporary three-VPC apply, VPN handshake/private reachability, public API, database restore, and second Ansible run (zero changes) passed; stack destroyed afterward
Full HD HEVC broadcast to Elemental with S3 archive	Local TS encoder, OBS instructions, SRT/MediaLive request generator	MediaLive input/channel and OBS HEVC profile were prepared; channel was not started, so live ingest and S3 archive remain unverified; resources deleted
Optional MSSQL	Not included	PostgreSQL backup/restore covers this application's recovery; it is not an MSSQL implementation

The repository holds implementation files and timestamped logs under docs/evidence/. It contains no cloud deployment evidence borrowed from another submission. For reproduction, start with the root README and the walkthroughs linked there.

2 Cross-platform storage and large media files

2.1 Protocol and filesystem decisions

SMB 3.1.1 provides native Windows file sharing and a Linux kernel CIFS client. The design requires authenticated access and encryption, with TCP 445 limited to private hosts or a VPN. A remote mount exposes files; the server retains ownership of its underlying filesystem. NFS is also suitable for some environments, but SMB reduces the Windows interoperability setup required here. iSCSI would expose a block device and would require different filesystem ownership and concurrent-access controls.

The Linux server uses XFS for large sequential media files, extent allocation, journaling and online growth. The Windows server can use NTFS. A 1 TB file does not require a single particular filesystem: ext4 can also meet the size requirement. The selection is based on the media workload and operational support. XFS cannot ordinarily be shrunk, so capacity planning must allow growth and backups.

Share permissions and filesystem permissions must both authorize the dedicated media user. The Samba configuration denies guests, permits that user only and requires SMB 3.1.1 encryption. For production, identity integration and ACL behavior should be tested with the actual Windows domain and Linux server configuration.

2.2 Windows server to Linux client

1. On Windows, use an existing data volume with sufficient capacity. Run storage/windows/setup-share.ps1 as administrator with the Linux client's address. The script creates the dedicated account, grants filesystem modification rights, creates the encrypted media share, disables SMB1 and restricts the firewall rule to that client.
2. On Linux, install cifs-utils, create /mnt/windows-media, and place the account in a root-owned /etc/smb-credentials file with mode 0600. Add the Windows host/domain when local account resolution requires it.
3. Mount using the following command after replacing the Windows address:

```
sudo mount -t cifs //WINDOWS_IP/media /mnt/windows-media \
-o credentials=/etc/smb-credentials,vers=3.1.1,seal,\
uid=1000,gid=1000,nosuid,nodev,noexec
```

4. Write from Linux and read the file on Windows. Write another file from Windows and read it on Linux. Compare SHA-256 hashes for a larger test file. Inspect Linux findmnt and CIFS debug data, and Windows Get-SmbConnection, for the negotiated dialect and encryption.
5. Adapt storage/linux/windows-share.mount.example into /etc/fstab. _netdev establishes network ordering; x-systemd.automount mounts on access; nofail lets the host boot if the server is unavailable. These settings do not make the share continuously available. Add RequiresMountsFor=/mnt/windows-media and an application readiness check for services that must use it.
6. Reload systemd, access the mount and repeat the integrity check. In the temporary AWS lab, both SMB directions negotiated encrypted SMB 3.1.1 and exchanged files successfully. Windows read a 1 TiB

sparse file created on Linux. A full Windows reboot left the persistent Z: mapping unavailable; the mapping must be re-established after reboot before production use. The Windows host and test share have since been deleted.

2.3 Linux server to Windows client

1. On Linux, prepare a dedicated XFS data volume, mount it at `/srv/media` and persist its mount by UUID. Identify the device and back up existing data before any physical-volume formatting. The supplied demo uses only a fresh temporary loop image.
2. Install Samba, create the `media` account, set its Samba password, and grant access to the share directory. Deploy `storage/smb.conf`, enable the Samba service and restrict TCP 445 to the Windows client's private address.
3. On Windows, run `storage/windows/map-linux-share.ps1 -LinuxServerIP LINUX_IP`. It prompts for credentials and maps the share to Z: using a persistent mapping and Windows credential storage.
4. Inspect `Get-SmbConnection`, perform bidirectional read/write/hash checks and test the mapping after a Windows reboot. Bidirectional exchange, encryption and large-file logical length passed. The mapping did not reconnect after reboot; troubleshoot Credential Manager/logon-session behavior and repeat this test before claiming persistent mapping support.

2.4 Local simulation and evidence

The local storage lab runs a Samba server and a Linux CIFS client on an internal Docker network. The client requires mount privileges inside Docker's Linux VM. Neither container is Windows; this simulation validates SMB mechanics and monitoring without claiming Windows interoperability.

`smb-mount.log` shows `vers=3.1.1`, `seal`, `nosuid`, `nodev` and `noexec`. `smb-roundtrip.log` records the written/read text and negotiated dialect `0x311`, with an encrypted AES128-GCM session. `smb-fio.log` records a 32 MiB direct-I/O scratch write with final `fsync`. Its throughput is a short lab measurement with possible server-side caching, not a physical-disk benchmark or a production capacity forecast.

The XFS demo creates a 2 TiB logical loop filesystem and a 1 TiB sparse file. The recorded result is `logical_bytes=1099511627776 allocated_blocks=0`. This establishes that XFS accepts a file exceeding 1 decimal TB. It does not write 1 TiB, prove physical free capacity or demonstrate sustained throughput at that scale. Real media storage needs actual allocation, quota headroom and a tested backup/restore path.

2.5 Monitoring and service-level alerts

A probe checks that the path is mounted, writes a small file, flushes it to the server, reads it back and exports success, latency and free/total bytes every ten seconds. An exporter reports Linux block-device metrics. On an actual Windows server, collect `PhysicalDisk` and `SMB Server Shares` counters through `PerfMon` or Windows exporter. On Linux, use `node exporter`, `iostat -xz`, CIFS statistics and bounded scratch-file tests. The Docker VM's disk metrics do not represent the Mac's physical drive directly.

Condition	Threshold and delay	Response
Share probe fails	Failure sustained for 30 seconds	Critical alert; check credentials, server, mount and backing disk
Probe latency grows	More than 2 seconds for 2 minutes	Warning; correlate network and storage metrics

Condition	Threshold and delay	Response
Available share space falls	Below 15% for 5 minutes	Warning; estimate time to full and expand/retire data
Probe exporter disappears	Scrape unavailable for 30 seconds	Critical alert; investigate monitoring and client
Linux device remains busy	Busy-time rate above 80% for 5 minutes	Warning; inspect queue, latency and workload

The five rules pass promtool validation. A deliberate unmount drove ShareUnavailable to firing; remounting resolved it. The saved JSON responses are alert-firing.json and alert-resolved.json. Production service objectives should be agreed separately, for example 99.9% monthly successful probes and a representative-file latency objective. Thresholds and a successful drill do not guarantee an SLA. External paging still needs Alertmanager integration, duplicate grouping, severity routing, resolved notifications and an agreed on-call runbook; no external notification was sent.

3 Docker application automation and recovery

3.1 Request path and network boundaries

The browser connects to Nginx on 127.0.0.1:8080. Nginx serves the frontend and forwards API calls to the Python application over the web network. The application connects to PostgreSQL over the internal data network. The frontend does not join the database network and the database has no published port. Only the backend joins both networks.

The application accepts a bounded operational note and writes it using parameterized SQL. It provides liveness independently from database readiness and returns a controlled 503 while the database is unavailable. It opens fresh database connections per request, allowing requests to resume after the database restarts. Nginx re-resolves the backend service name so backend recreation does not permanently leave it pointing at a stale container address.

3.2 Automated setup and credential handling

```
bash scripts/setup.sh
docker compose up -d --build
python3 scripts/smoke.py
```

The setup script generates local random passwords if they do not exist. The host secrets/ directory is owner-only. The files are read-only bind mounts for authorized containers; a readable file mode inside that protected directory supports the non-root application on Linux and macOS. They are excluded from Git. Compose file secrets do not provide encrypted storage, managed rotation or a cloud vault. The AWS server procedure instead uses encrypted Ansible Vault values and a restricted application credential file.

Compose waits for database health before starting the backend and for backend health before starting Nginx. The backend runs without root, has a read-only filesystem, uses a temporary /tmp, drops capabilities and prevents privilege escalation. Logging is bounded to three 10 MB files per application service. The readiness endpoint tests a database query; liveness only tests the process. A failed health check does not itself restart a container under ordinary Compose.

3.3 Resource allocation

Service	CPU quota	RAM limit	Storage and process handling
PostgreSQL	1 CPU	512 MiB	Named data volume; 150-process limit
Backend	0.75 CPU	256 MiB	Read-only image, temporary / tmp; 80-process limit
Nginx/frontend	0.25 CPU	96 MiB	Static assets and bounded logs; 50-process limit

These are starting limits for a small assessment workload, not measured production sizing. CPU quotas limit runnable CPU time; they do not reserve a dedicated core. RAM limits contain a service's memory use but may cause OOM termination. PostgreSQL's named volume is persistent across container recreation but is not a fixed storage quota. Monitor Docker's available disk space and impose a real storage budget or host filesystem quota when deploying. Leave operating-system and Docker VM headroom; the observed VM had approximately 8 GB assigned.

3.4 Recovery behavior and executed tests

All three services use `restart: unless-stopped`. If a main process exits unexpectedly, Docker restarts its container once the daemon is running. After host shutdown, application recovery also depends on Docker starting. Docker Desktop must be configured to start at macOS login for this lab. The restart policy does not repair a failed host, restore erased storage or guarantee zero interruption.

The executed checks killed the backend main process, recreated the backend, stopped/restarted the database, and recreated the database container. The saved marker remained readable after recreation through the named volume. During the database outage, the liveness endpoint stayed available and API calls returned 503. `app-recovery.json` records these passes. A separate disposable 64 MiB container exceeded its memory budget and was recorded as `OOMKilled=true`, exit 137. This proves cgroup memory containment; a full host RAM-exhaustion or power-failure drill was not performed.

PostgreSQL backup uses `pg_dump -Fc` through `scripts/backup.sh`. A test restored the dump into a temporary database and compared the record count, then removed that temporary database. Both databases contained three records in the captured run. This is a demonstrated PostgreSQL restore, not an implementation of the optional MSSQL task. For production, schedule backups outside the application container, encrypt and retain them off-host, and test recovery time and acceptable data loss. A persistent volume alone is not a backup.

4 Node-RED and S3 multipart transfer

4.1 Copy workflow and automation

The Node-RED inject node invokes an internal HTTP worker, which performs a server-side multipart copy. The result node displays status and completion information, and a catch node captures flow errors. Node-RED's editor is exposed only on localhost for this demonstration. The worker accepts configured source/destination values rather than caller-supplied arbitrary code or credentials.

```
docker compose -f compose.s3.yaml up -d --build
python3 scripts/verify.py s3
```

The test creates a 14,000,000-byte object in the local Moto emulator and copies it between two buckets. Open Node-RED on port 1880 and inject **Copy configured object** to repeat the operation. `nodered-flow.log` records an actual flow trigger and a successful HTTP 200 response, rather than only a direct worker call.

The transfer engine can also run as a standalone script:

```
python3 s3/multipart_copy.py SOURCE_BUCKET OBJECT_KEY DESTINATION_BUCKET \
  --part-mib 64 --concurrency 8
```

Install its requirements first. The local Compose file provides emulator-only credentials and an emulator endpoint. On AWS, omit that endpoint and use the SDK's credential chain, preferably a short-lived IAM role. Adapt `s3/iam-policy.example.json` to the exact buckets; customer-managed KMS encryption requires corresponding key permissions.

4.2 Multipart limits and failure handling

The current S3 limits are 10,000 parts, with each non-final part between 5 MiB and 5 GiB. The documented object maximum is 48.8 TiB. The planner therefore derives an effective part size as `max(preferred_part_size, ceil(object_size / 10000))` and rejects sizes outside the limit. A 1 TiB object with 64 MiB preferred parts automatically grows its parts to remain within the maximum count. Maximum object size and optimal concurrency are separate concerns; 32 concurrent requests is this implementation's bounded ceiling, not a claim about maximum possible S3 throughput. [S3 multipart limits](#).

The engine HEADs the source, pins its version when available, and supplies an ETag precondition on each part. It creates a multipart upload, issues bounded concurrent `UploadPartCopy` calls, records the returned ETags, sorts parts by number, completes the upload and checks destination size. Content type and custom metadata are preserved. Tags, all other system metadata and destination encryption policy need separate consideration; this script is not a universal clone of every object property.

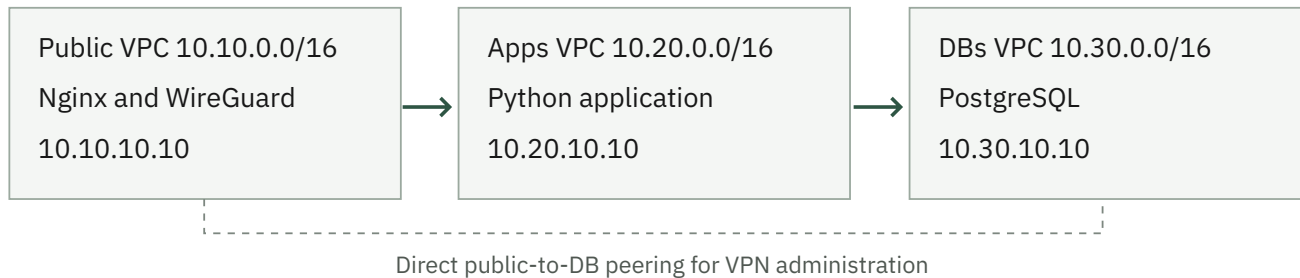
SDK retries use standard exponential retry behavior. If a part fails, the worker waits for outstanding threads before aborting, avoiding uploads arriving after cleanup. A crash that prevents cleanup can leave orphaned parts, so the Terraform buckets include a two-day incomplete-upload lifecycle rule. Zero-byte objects take a separate copy path. Logs contain UTC time, part completion, byte counts, duration, throughput and cleanup events without credential values.

4.3 Executed evidence and limits

The local test copied the object in three parts: 5,242,880 bytes, 5,242,880 bytes and 3,514,240 bytes. A separate SHA-256 check verified the source and destination bytes. Boundary tests exercised the planner through the current maximum; a synthetic failed part confirmed abort behavior. These tests passed. The emulator's reported throughput is not an AWS performance measurement, and no multi-terabyte object was transmitted. Real S3 permissions, KMS handling, regional behavior, network conditions and service-side integrity must still be tested.

5 Three-tier AWS infrastructure and Ansible

5.1 Three VPCs and direct routing



The architecture uses three separate VPCs: public `10.10.0.0/16`, apps `10.20.0.0/16`, and DBs `10.30.0.0/16`. Each workload subnet is `/24`; application and database VPCs also have public egress subnets for their NAT gateways. The public EC2 instance runs Nginx and WireGuard. Application and database instances have private addresses only.

Direct peerings join public↔apps, apps↔DBs and public↔DBs. Routes are declared on both sides. VPC peering is non-transitive, and a VPC cannot use a peered VPC's NAT or internet gateway. Each private VPC therefore has its own NAT egress for installation and updates. [AWS peering behavior](#).

Security groups allow public HTTP to Nginx, bootstrap SSH and WireGuard from the configured administrator `/32`, backend port 8000 only from the gateway address, and database port 5432 only from the application address. Private SSH is allowed only from the gateway's private address. The WireGuard gateway masquerades client traffic to that address, and EC2 source/destination checking is disabled for forwarding. VPN administrators use SSH and a local Unix-socket database session; the VPN does not grant unrestricted public database access.

5.2 Deployment sequence and configuration automation

1. Configure temporary AWS credentials. Fill `infra/terraform.tfvars` from its example with region, public SSH key and current administrator `/32`. Keep the private key, state and plan files outside Git.
2. Initialize and validate Terraform, review a saved plan, then apply only with an account and a reviewed budget. A temporary deployment in `eu-west-1` created the three tagged VPCs, NAT gateways, Linux hosts, storage buckets and Windows test host. The resources were removed with Terraform after validation.
3. Export `ansible_inventory` from Terraform to the Ansible inventory file. Populate and encrypt `vault.yml` with the database password and the WireGuard client public key.
4. Create a client tunnel in the macOS WireGuard app. Configure only the reachable gateway first using `ansible-playbook ... --limit gateway`. Obtain its public VPN key and fill the client endpoint and the three VPC routes.
5. Activate the VPN and confirm a handshake and private SSH access. Run the full playbook to install PostgreSQL, create its role/database, deploy the Python application in a virtual environment and enable its systemd service.
6. Run Ansible again to assess idempotence. Verify public write/read through Nginx, private routing, absence of private-host public IPs, and loss/restoration of administrative access when the VPN is disconnected/reconnected.

The playbook uses package, file, template, database and service modules, with handlers for configuration changes. Server-side credentials use restrictive file permissions and `no_log`. The application service has `Restart=on-failure`, `MemoryMax=512M` and `CPUQuota=100%`. The database and proxy services are enabled at boot. Exact commands and client configuration are in `infra/DEPLOY.md`.

5.3 Validation boundary and operational tradeoffs

Terraform format/validation and a reviewed create-only plan passed. The deployed VPN handshake and private HTTP readiness check passed; the app and database hosts had no public address. Public API write/read passed, and a second full Ansible run reported zero changes on all three Linux hosts. A backup was restored to a temporary database, its marker row verified, and the temporary database dropped. This was a brief single-AZ acceptance lab, not a production reliability or security audit. All lab resources were destroyed and tagged inventory checks returned no VPCs, running instances, NAT gateways, or buckets; the exact MediaLive channel and input returned DELETED, and test IAM roles were absent.

The design has one instance per tier in one availability zone and supplies no multi-AZ high availability. HTTP is acceptable only for this isolated demonstration; production needs TLS, authenticated application access, database transport protection, centralized logs and a tested recovery design. Two NAT gateways, EC2, public addresses, EBS and transfer may all incur charges. Confirm current regional pricing before deployment. The S3 buckets block public access, enable versioning/encryption and clean abandoned multipart uploads; non-empty buckets are not automatically destroyed by this configuration.

6 HEVC broadcast and Elemental archive

6.1 Codec, transport and bitrate selection

The proposed route is OBS → encrypted SRT carrying MPEG-TS/HEVC → AWS Elemental MediaLive → MPEG-TS archive objects in S3. HEVC is the video codec, SRT is the transport and MPEG-TS is the container. MediaLive's RTMP input supports H.264, whereas its SRT listener input supports HEVC. Archive output supports HEVC in TS sent to S3. [Input codecs](#), [Archive output](#).

The target is 1920×1080, 25 fps, 12 Mbps HEVC video and AAC stereo at 192 kbps/48 kHz. GOP/keyframe interval is two seconds. The bitrate remains at the assessment's minimum rather than reducing it. The optional BPP calculation is:

```
BPP = video bitrate / (width × height × frames per second)
     = 12,000,000 / (1920 × 1080 × 25)
     = 0.23148 bits per pixel per frame
```

BPP normalizes the bitrate but cannot guarantee perceptual quality. Motion, noise, encoder quality, GOP and chroma subsampling also matter. Video and audio together are nominally 12.192 Mbps before container/transport overhead, or approximately 5.49 GB per archive hour. A stable upstream connection with at least 20 Mbps headroom is a planning target; the actual link must be measured.

6.2 Local media verification

`streaming/encode.sh` generates ten seconds of moving Full HD test video with noise audio, encodes HEVC at a configured 12 Mbps with AAC at 192 kbps and writes MPEG-TS. The probe confirms HEVC Main, 1920×1080, 25 fps, AAC stereo and MPEG-TS. Packet-level measurement records video at 12,025,571 bps and audio at 194,565 bps over the short sample; whole-container bitrate is 12,524,162 bps. Configured and observed rates are recorded separately in `streaming-summary.json`.

The local test used FFmpeg inside Docker. OBS was installed and its Apple Silicon build exposes a VideoToolbox HEVC encoder, but the configured custom FFmpeg output was not successfully verified. A real AWS MediaLive input and channel configuration were accepted by the service, but the channel remained IDLE and was deleted without being started. No OBS-to-AWS ingest or S3 archive was produced. The `.ts` file is reproducible and excluded from Git to keep the source package small.

6.3 MediaLive and OBS deployment steps

1. Prepare an encrypted/private S3 archive bucket and a MediaLive role with permissions for its archive prefix and SRT secret. Create a Secrets Manager secret containing a 10-79 character passphrase as a plaintext secret value, not a JSON object. Secrets Manager encrypts that stored value. Create an input security group allowing the sender's current public /32.
2. Generate `CreateInput` and `CreateChannel` JSON using `streaming/render-config.py`, supplying real bucket, role ARN, secret ARN and input security group. Create the SRT listener input, record its actual ID and allocated endpoint, then regenerate the channel request with that ID. The listener requires AES encryption and uses port 5050. [SRT listener setup](#), [Passphrase prerequisites](#).
3. Create a single-pipeline MediaLive channel using HD HEVC input specification and the target video/audio settings. Its Archive group rolls TS objects into the S3 recording prefix every 60 seconds. Single pipeline is a cost-conscious test setting, not a redundant production design. Start the channel and wait for RUNNING.
4. In OBS on macOS, set video canvas/output to Full HD at 25 fps and choose a supported HEVC encoder, preferably Apple VideoToolbox when exposed by the installed build. Configure 12000 kbps video, two-second keyframes and AAC 192 kbps.
5. Use the SRT caller URL with the allocated endpoint, negotiated passphrase, AES-256 key length and one-second latency. Stream key is empty. OBS uses latency values in microseconds, while the MediaLive input configuration uses milliseconds. A URL containing the passphrase must be redacted from screenshots and logs. [OBS SRT guide](#).
6. If the installed OBS build does not expose HEVC for normal streaming, use its custom FFmpeg output-to-URL path, MPEG-TS and HEVC, if supported. This path is activated through Start Recording despite writing to a network endpoint. Verify the actual encoder; silently substituting H.264 does not satisfy the requirement.
7. Inspect MediaLive input/output metrics and alerts, wait for archive rollover, list actual S3 objects and download a generated `.ts` file. Probe it and measure the elementary-stream rates. Save redacted OBS settings, running-channel metrics and the real archive evidence.
8. Stop the channel after testing and remove unused billable resources while preserving submission access. Exact CLI steps, policy examples and remaining checks are in `streaming/AWS.md` and `streaming/OBS.md`.

The generated request files pass SDK schema validation. In the acceptance run, AWS also accepted creation of the SRT listener input and single-pipeline HEVC/archive channel configuration. This confirms service-side acceptance of those settings, not successful ingest, output encoding or archive delivery. The OBS output path still requires correction and a short live-stream retest.

7 Evidence index and remaining acceptance checks

Evidence	Demonstrated result
smb-mount.log, smb-roundtrip.log	Actual encrypted SMB 3.1.1 mount and read/write
xfs-sparse.log	XFS accepts a 1 TiB sparse file
storage-alert-rules.log, alert JSON files	Five valid rules; unavailable-share alert fires and resolves
app-smoke.log, app-recovery.json	Application round trip, recovery and volume persistence
oom-limit.log, postgres-restore.json	Isolated OOM containment and PostgreSQL restore
s3-tests.log, nodered-flow.log	Multipart integrity, cleanup and actual Node-RED execution
terraform-validate.log, ansible-syntax.log	Local infrastructure/configuration checks
hevc-probe.log, streaming-summary.json	Local Full HD HEVC/AAC media and measured rates
aws-s3-multipart.log	Real three-part S3 copy, integrity hash match and cleanup
aws-acceptance.log, aws-ansible-idempotence.log, aws-database-restore.log	Cloud API, repeated configuration run and restore verification
aws-linux-smb.log, aws-windows-smb.log, aws-windows-largefile.log	Encrypted bidirectional Windows/Linux SMB and 1 TiB logical file
aws-windows-reboot.log	Reboot check showing the Windows drive mapping did not reconnect
aws-cleanup.log	Terraform destroy completion for the temporary stack
medialive-schema.log	MediaLive request shape checks

The remaining acceptance items are to fix Windows drive reconnection after reboot and complete one OBS-to-MediaLive test that confirms HEVC ingest and an actual S3 archive object. Current cloud resources have been deleted to control cost; repeating those checks requires a new temporary deployment and budget review. The tested S3 multipart transfer and AWS infrastructure stack are not currently running.

8 References and provenance

The Arabic assessment document supplied by the candidate defines the requirements. The supplied [public reference repository](#) was read to compare scope and deployment considerations; its implementation and evidence were not copied. The [Drive archive](#) supplied the IBM Plex Sans Arabic font family, whose license remains with the repository assets.

Primary technical references appear beside the relevant decisions. For commands and implementation details, see the repository files and the linked AWS, OBS, Terraform and WireGuard documentation. Submission instructions require keeping shared materials accessible during the company's review period, stated as up to seven days after submission.